Quantstamp

Kiln DeFi

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DeFi Aggregator
Timeline	2024-02-05 through 2024-02-15
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	README file
Source Code	• https://github.com/kilnfi/defi-integrations #db9844e
Auditors	• Valerian Callens Senior Auditing Engineer • Poming Lee Senior Auditing Engineer • Jeffrey Kam Auditing Engineer • Gelei Deng Auditing Engineer

Documentation quality	High
Test quality	Medium
Total Findings	29 Fixed: 6 Acknowledged: 19 Mitigated: 4
High severity findings	1 Fixed: 1
Medium severity findings	5 Acknowledged: 3 Mitigated: 2
Low severity findings	9 Fixed: 2 Acknowledged: 6 Mitigated: 1
Undetermined severity findings	6 Fixed: 2 Acknowledged: 3 Mitigated: 1
Informational findings	8 Fixed: 1 Acknowledged: 7

Summary of Findings

This audit focused on Kiln DeFi, a protocol allowing Integrators to deploy ERC-4626 vaults that can interact with DeFi protocols via a set of connectors managed by Kiln. End-users can deposit funds to these vaults, that will be further deposited on integrated protocols to generate yield. Integrators will handle management fees (on deposit and withdrawal) and performance fees (on rewards) collected at the level of their vault.

Regarding the issues, we mainly highlighted an incorrect accounting of funds, opportunities to strengthen the integration with DeFi protocols if something unexpected happens, a lack of liquidity reward support, and potential Denial of Service vectors. We also have six "Undetermined" severity issues, mainly related to the function `IConnector.totalAssets()` which plays a critical role in the system and could be the source of multiple ways to negatively impact the system.

Regarding the project quality, the system and the codebase are well documented. Regarding the test suite, it consists of unit tests and end-to-end tests. Happy and unhappy paths are tested, except for the cases where integrated protocols would not work as expected. Coverage results are close to 100% for all contracts in scope, which is very good, except for the contracts integrating with Morpho, where we obtain a coverage score of `0.00%` even if tests are defined.

Our team frequently interacted with the Kiln team to clarify the code and its expected behavior. Their active engagement in answering our questions was crucial to complete the audit.

Fix review update

The client has either fixed, mitigated, or acknowledged the issues in this report. The test suite has been updated, and despite a minor decrease, the coverage results for the Vault and Connectors contracts remain satisfactory. Additionally, enhancements have been made to the documentation to clarify the management of additional rewards from integrated protocols.

ID	DESCRIPTION	SEVERITY	STATUS
KIDF-1	Incorrect Accounting Update of `$._lastTotalAssets` when Performance Fees Are Collected From the Integrated Protocol	High	Fixed
KIDF-2	No Check to Make Sure that the Amounts Transferred and Received Match the Expected Amounts	Medium	Mitigated
KIDF-3	IConnector Operations Can Fail (Silently or Not) when Executed on Integrated Protocols, Leading for Instance to Race Conditions Opportunities	Medium	Acknowledged
KIDF-4	Liquidity Mining Rewards From Integrated Protocols Cannot Be Claimed	Medium	Mitigated
KIDF-5	Privileged Roles and Centralization Risks	Medium	Acknowledged
KIDF-6	Potential Shares Burning After Rewards	Medium	Acknowledged
KIDF-7	Usage of `view` Functions that Can Revert	Low	Mitigated
KIDF-8	Missing Input Validation	Low	Fixed
KIDF-9	The Function `_dispatchFees()` Is Exposed to Denial of Service	Low	Acknowledged
KIDF-10	Pending Fees Are Not Dispatched when the List of Fee Recipients Is Updated	Low	Acknowledged
KIDF-11	Upgradability	Low	Acknowledged
KIDF-12	Integration Requirements for Smart Contracts Are Not Documented	Low	Acknowledged
KIDF-13	Ambiguous Events when Fees Are Collected and Dispatched	Low	Acknowledged
KIDF-14	User May Not Be Able to Deposit Tokens with Non-Standard Decimals	Low	Acknowledged
KIDF-15	Function `dispatchFees()` Is Not Protected Against Reentrancies	Low	Fixed
KIDF-16	Risks of Supporting Non-Standard Erc-20 Tokens	Informational	Acknowledged
KIDF-17	Complexity of the Function `_setFeeRecipients()` Can Be Reduced to Save Gas	Informational	Acknowledged
KIDF-18	Users Cannot Withdraw Same Amount of Assets Due to Precision Loss	Informational	Acknowledged
KIDF-19	Management Fee Can Be Bypassed Due to Precision Loss	Informational	Acknowledged
KIDF-20	Supply of Native Token May Not Be Supported for Compound V2 Connector	Informational	Acknowledged

ID	DESCRIPTION	SEVERITY	STATUS
KIDF-21	**Salt Generation Mechanism to Create Vaults Could Be Tricked**	Informational	Acknowledged
KIDF-22	**Redundant Operations when Interacting with Morpho**	Informational	Acknowledged
KIDF-23	**Consolidation of Best Practices**	Informational	Fixed
KIDF-24	**If `totalAssets()` Decreases, the Operations `deposit()`, `mint()`, `withdraw()` and `redeem()` Will Revert**	Undetermined	Mitigated
KIDF-25	**Read-only Reentrancy Can Happen by Manipulating `totalAssets()`**	Undetermined	Acknowledged
KIDF-26	**`totalAssets()` May Provide an Outdated Amount of Assets for `CompoundV2` Connector**	Undetermined	Fixed
KIDF-27	**Possibility to repeatedly perform opposite actions at almost no cost**	Undetermined	Acknowledged
KIDF-28	**Addresses with the Role `PAUSER` Can Unpause Contracts Even if it Should Be Restricted to Addresses with the Role `UNPAUSER`**	Undetermined	Fixed
KIDF-29	**Integrators Can Remove Kiln From the Fee Recipients**	Undetermined	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.

2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.

2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope was limited to the files in the folder `src/*` at the commit `db9844e`

In particular, the following files were added to the codebase in a later commit and are not in the scope of this audit:

- src/connectors/MetamorphoConnector.sol
- src/connectors/MorphoBlueConnector.sol
- src/connectors/VenusConnector.sol
- src/interfaces/ISanctionsList.sol

Findings

KIDF-1

Incorrect Accounting Update of `$._lastTotalAssets` when Performance Fees Are Collected From the Integrated Protocol

• High Fixed

Update

The client updated the accounting accordingly.

Update

Marked as "Fixed" by the client. Addressed in: `741b2aa9cd15d3a708399d52ece4ed61a8d56abc` , `f6e0bc40497194e80d0198c9b80f027d0c205ad4` . The client provided the following explanation:

> Merged with this PR ⇒ https://github.com/kilnfi/defi-integrations/pull/24

File(s) affected: `Vault.sol`

Description: When performance fees are collected from the integrated protocol via the function `collectPerformanceFees` , the amount of `$._lastTotalAssets` (that should represent the number of assets deposited on the integrated protocol) is updated with the value `_totalAssets` . This value is incorrect and higher than the reality since performance fees have been collected from the integrated protocol. The new value should be `totalAssets()` . This can negatively impact the system since the function `_currentPerformanceFeeAmount()` will revert due to an underflow until enough yield is generated to close the gap of the incorrect accounting.

Exploit Scenario: Consider the following test case

```
// test the specific case of collecting performance fee and deposit again.
    function test_custom_collectPerformanceFees(uint256 rewards) public {
        uint256 depositAmount = 101; // avoid management fee with 1
        _singleDeposit(alice, depositAmount);
        rewards = 100;
        protocol.addRewards(IERC20(address(asset)), rewards);

        admin.impersonate();
        vault.collectPerformanceFees();
        accounts.stopImpersonate();

        // deposit again. Should revert
```

```
        _singleDeposit(alice, depositAmount);
    }
```

The error is:

```
Encountered 1 failing test in test/CustomVault.t.sol:CustomVaultTest
[FAIL. Reason: panic: arithmetic underflow or overflow (0x11); counterexample:
calldata=0x0fe4534100000000000000000000000000000000000000000000000000000000000000000 args=[0]]
test_custom_collectPerformanceFees(uint256) (runs: 0, μ: 0, ~: 0)
```

Recommendation: Consider updating the accounting accordingly.

KIDF-2

No Check to Make Sure that the Amounts Transferred and Received Match the Expected Amounts

Medium | Mitigated

Update

1. Since the client acknowledged "KIDF-16 Risks of Supporting Non-Standard Erc-20 Tokens", we should not expect to receive a different amount in `Vault._deposit()` when tokens are transferred from the owner;
2. The function `Vault._deposit()` executes a deposit with the exact amount that was received by the contract from the `owner`. However, depending on the implementation of the integrated protocol, it is not certain that depositing `n` tokens will result in increasing `totalAssets()` by `n`. The client acknowledged the absence of this check.
3. The client updated the function `Vault._withdraw()` to make sure that the exact amount of assets received after withdrawing from the integrated protocol is transferred to the user. However, depending on the implementation of the integrated protocol, it is not certain that withdrawing `n` tokens from the integrated protocol will result in receiving `n` tokens in the contract.
4. The client updated the function `Vault.collectPerformanceFees()` to make sure that the exact amount of assets received after withdrawing from the integrated protocol is added to the `pendingPerformanceFee`. However, depending on the implementation of the integrated protocol, it is not certain that withdrawing `n` tokens from the integrated protocol will result in receiving `n` tokens in the contract. Also, the following expression `$._collectablePerformanceFees = 0;` could be incorrect if less than `n` tokens were received by the contract.

Update

Marked as "Fixed" by the client. Addressed in: `4d6061eeefacac199602e1e5e9ca296b71ac5ac8`. The client provided the following explanation:

> The fix was already done for the deposit. However, the increase of `totalAssets()` is not checked. It's recommended to use a router to do so.

File(s) affected: `Vault.sol`

Description: At several locations in the code, token amounts are transferred. No check is performed to make sure that the amount requested has been received once the operation is executed. It is the case in:

1. `Vault._deposit()` when tokens are transferred from the owner;
2. `Vault._deposit()` when tokens are deposited to the integrated protocol;
3. `Vault._withdraw()` when tokens are withdrawn from the integrated protocol;
4. `Vault.collectPerformanceFees()` when fees are collected from the integrated protocol;

Note that some of these checks could also be implemented at the connector level, in the functions `IConnector.deposit()` and `IConnector.withdraw()`.

Recommendation: Consider adding checks to make sure that amounts received after a transfer match what is expected.

KIDF-3

IConnector Operations Can Fail (Silently or Not) when Executed on Integrated Protocols, Leading for Instance to Race Conditions Opportunities

Medium | Acknowledged

Update

> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > It should be resolved using a router that will check the `totalAssets()` increase. We won't include a direct solution in the `Vault` codebase.

File(s) affected: `Vault.sol`

Description: Executing the functions `IConnector.deposit()` and `IConnector.withdraw()` may fail depending on multiple reasons on the integrated protocols. For instance:

1. the underlying market may be paused;
2. the supply cap can be reached;
3. there may not be enough liquidity available to withdraw, because funds are temporarily unavailable (borrowed) or the underlying protocol suffered an attack and lost a significant portion of users' funds;

As a result, especially for the scenarios 2 and 3, users may be subject to race conditions.

In specific cases, such as for the integration with `Compound V2` here, all functions `getAccountSnapshot()`, `mint()`, and `redeemUnderlying()` return an error code to indicate if the operation was a success or not. Since the values returned by these functions are ignored, these functions can silently fail without having any effect on the calling functions. Such silent failure can lead to situations where funds that failed to deposit into the protocol are locked in the `Vault` contract. Furthermore, this could also dilute the value of each share in the vault because `totalAssets()` provided by the underlying protocol does not increase, but new shares are minted. Note also that the current codebase of `Compound V2` only returns an error value corresponding to successful operations, but contracts are upgradeable and this may change in the future.

Recommendation: Consider:

1. using the values returned by the integrated protocol and enforce proper error handling. Specifically for Compound V2, the code should revert if the returned error code is not zero.
2. assessing if temporary errors when depositing to and withdrawing from the integrated protocols should lead or not to temporarily pausing the associated connector.
3. Documenting to end-users that their funds are exposed to the risks of the integrated protocols, and front-running if the supply cap of the integrated protocol is almost reached or the liquidity available to be withdrawn is limited.

KIDF-4
Liquidity Mining Rewards From Integrated Protocols Cannot Be Claimed

Medium | Mitigated

> **Update**
> The client documented how rewards are managed in the `README` file. They also added an abstract method to claim rewards from the integrated protocols, to be executed by a new role `CLAIMER`. The implementation of this method is done at the connector level, and only for the `Compound V2` integration. Rewards are claimed, swapped into the deposited assets, and deposited back to the integrated protocol. However, we would like to highlight that this new process adds new risks that should be carefully addressed:
>
> 1. the connector should only approve the amount of rewards received, instead of a max approval.
> 2. slippage prevention should be enforced inside the `payload` parameter, as well as double-checked at the connector level based on the variation of the balance pre and post-operation.
> 3. some aggregators and/or swappers can have a callback mechanism, such as `1Inch V5`. The `1Inch V5` router exposes a `swap()` function, which allows the caller to specify the swap executor and receive a call from the router during the swap. Currently, the functions of the contract Vault are protected against reentrancies but the team should be careful for next upgrades.
> 4. the behavior of `swapTarget` may change if it is upgradeable. Careful monitoring of this address would help prevent any unexpected behavior.

> **Update**
> Marked as "Fixed" by the client. Addressed in: `ba3a3a6ef81016b1e90088e6256b8dfa4a0cca92`. The client provided the following explanation:
>
> > See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/36

File(s) affected: `IConnector.sol`

Description: When depositing tokens to protocols like Aave, the liquidity provider often earns protocol rewards, in addition to the yield, as a part of their incentives to interact with the protocol. In Aave V2 and V3, certain pools provide liquidity mining rewards for supplying tokens to the Aave protocol, where the provider can claim the rewards by calling `claimRewards()` (see here). For

Compound V3, there is a similar notion of liquidity rewards in the form of `COMP` tokens, which can also be claimed by the liquidity provider. The situation for Morpho is a bit different, where the rewards include both the `MORPHO` tokens but also the underlying protocol rewards such as `COMP` (see here). However, there is currently no way for the Vault to claim these rewards, and even if these rewards are claimed, there is no mechanism to distribute these rewards to the Vault's shareholders proportionally.

Similarly, the same issue occurs for any airdrop rewards claimable by using the underlying protocols, even if the likelihood of such events is lower.

Recommendation: Consider adding a rewards-claiming mechanism for each of the connectors. Furthermore, the team should clearly outline how these rewards will be distributed to the users or the team, and implement the features accordingly.

KIDF-5 Privileged Roles and Centralization Risks

Medium Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > All the roles are documented in the README file.

File(s) affected: `VaultFactory.sol, Vault.sol, ConnectorRegistry.sol, VaultUpgradeableBeacon.sol`

Description: These are the privileged operations identified during the audit:

In `VaultFactory.sol`:

- Addresses with the `DEFAULT_ADMIN_ROLE` can:
  - Grant or revoke the role `DEPLOYER_ROLE`;
  - transfer this role in two delayed steps;
  - renounce its role in two steps via the function `renounceRole()`;
- Addresses with the `DEPLOYER_ROLE` can:
  - deploy a new vault via the the function `createVault()`;
  - Renounce this role;

In `Vault.sol`:

- Addresses with the `DEFAULT_ADMIN_ROLE` can:
  - Grant or revoke the role `FEE_MANAGER_ROLE`;
  - transfer this role in two delayed steps;
  - renounce its role in two steps via the function `renounceRole()`;
- Addresses with the `FEE_MANAGER_ROLE` can:
  - collect performance fees via the the function `collectPerformanceFees()`;
  - set the fee recipients to any arbitrary addresses in any proportions via the function `setFeeRecipients()`;
  - set the management fee to any value between `0%` and `35%` via the function `setManagementFee()`;
  - set the performance fee to any value between `0%` and `35%` via the function `setPerformanceFee()`;
  - Renounce this role;

In `ConnectorRegistry.sol`:

- Addresses with the `DEFAULT_ADMIN_ROLE` can:
  - Grant or revoke the roles `PAUSER_ROLE`, `UNPAUSER_ROLE`, `FREEZER_ROLE`, `CONNECTOR_MANAGER_ROLE`;
  - transfer this role in two delayed steps;
  - renounce its role in two steps via the function `renounceRole()`;
- Addresses with the role `PAUSER_ROLE` can:
  - pause the requests of the address associated with a given connector name for:
    - an infinite amount of time via the function `pause()`;
    - a given amount of time via the function `pauseFor()`;
  - Renounce this role;
- Addresses with the role `UNPAUSER_ROLE` can:
  - unpause the requests of the address associated with a given connector name via the function `unpause()`;
  - Renounce this role;
- Addresses with the role `FREEZER_ROLE` can:
  - freeze the updates and removal of the associations between an address and a connector name via the function `freeze()`;
  - Renounce this role;
- Addresses with the role `CONNECTOR_MANAGER_ROLE` can:
  - set, update, and remove the associations between an address and a connector name via the functions `add()`, `update()`, and `remove()`;
  - Renounce this role;

In `VaultUpgradeableBeacon.sol`:

- Addresses with the `DEFAULT_ADMIN_ROLE` can:

    - Grant or revoke the roles `PAUSER_ROLE`, `UNPAUSER_ROLE`, `FREEZER_ROLE`, `IMPLEMENTATION_MANAGER_ROLE`;
    - transfer this role in two delayed steps;
    - renounce its role in two steps via the function `renounceRole()`;
- Addresses with the role `PAUSER_ROLE` can:
    - pause the requests of the current `implementation` address for:
        - an infinite amount of time via the function `pause()`;
        - a given amount of time via the function `pauseFor()`;
    - Renounce this role;
- Addresses with the role `UNPAUSER_ROLE` can:
    - unpause the requests of the address associated with a given connector name via the function `unpause()`;
    - Renounce this role;
- Addresses with the role `FREEZER_ROLE` can:
    - freeze the updates of the current `implementation` address via the function `freeze()`;
    - Renounce this role;
- Addresses with the role `IMPLEMENTATION_MANAGER_ROLE` can:
    - updates of the current `implementation` address via the function `upgradeTo()`;
    - Renounce this role;

Recommendation: These roles and the associated privileges should be documented to users. Also, key management should follow the latest best practices in security.

KIDF-6 Potential Shares Burning After Rewards

Medium | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> This issue is from the ERC-4626 standard design; we won't move away from the standard.

File(s) affected: `Vault.sol`

Description: In `Vault.sol`, there are two ways of withdrawing: `withdraw()` and `redeem()`, where the former one withdraws assets directly, and the second withdraws shares in exchange for assets. The problem happens in the function `withdraw()`. When the absolute value of assets is larger than the number of shares (`totalAssets() > totalSupply()`), withdrawing `1` single token through `withdraw()` function will cost `1` share. An adversary can repeatedly do this to burn the shares to `1`. If there are `a` tokens and `b` shares, and `a>b`, all controlled by the adversary, then the adversary can withdraw `a−b+1` tokens and burn all the shares to `1`. The direct consequence of the above issue is that in the end, the token-share ratio becomes `a−b+1:1`, which is not ideal. Any other users who want to deposit will have to deposit a large amount of tokens to get at least `1` share, and potentially suffer from huge precision loss. We provide a test case below. Note that the test case can pass without any issues.

Exploit Scenario: A test case is provided below for demonstration.

```
    function _singleWithdraw(address user, uint256 amount) internal {
        user.impersonate();
        vault.withdraw(amount, user, user);
        accounts.stopImpersonate();
    }
    // other utility functions
    // ...
    function test_share_burning() public {
        // step 1: create a scenario where the totalAssets() > totalSupply()
        // Alice makes a deposit
        _singleDeposit(alice, 5);
        // then add rewards of 5.
        protocol.addRewards(IERC20(address(asset)), 5);
        // Now there are 10 assets and 5 shares.
        uint256 obtainedShares = vault.balanceOf(alice);
        expect(obtainedShares).toEqual(5);
        // Alice can then burn the shares to 1 by withdrawing 4 times
        _singleWithdraw(alice, 1);
        _singleWithdraw(alice, 1);
        _singleWithdraw(alice, 1);
        _singleWithdraw(alice, 1);
        // Note that now the shares have been burned (Vault balance should be 6, yet 1 share only.)
        uint256 aliceSharesAfterBurn = vault.balanceOf(alice);
        expect(aliceSharesAfterBurn).toEqual(1);
```

```
        expect(vault.totalAssets()).toEqual(6);
        // Bob tries to deposit 3 assets, but unfortunately cannot get any shares in return.
        uint256 bobShares = vault.balanceOf(bob);
        expect(bobShares).toEqual(0);
        _singleDeposit(bob, 3);
        bobShares = vault.balanceOf(bob);
        expect(bobShares).toEqual(0);
        // assert total assets and shares in the vault
        expect(vault.totalAssets()).toEqual(9);
        expect(vault.totalSupply()).toEqual(1);
    }
```

Recommendation: We recommend the team to consider if this is the intended behavior. If not, always ensure that `share >= asset` so that whenever a user performs a withdrawal, the relative ratio can be maintained. Setting `1 asset = 10 shares` would be a way of doing so.

KIDF-7 Usage of `view` Functions that Can Revert

Low Mitigated

Update

The client fixed points 1 and 3.

Update

Marked as "Fixed" by the client. Addressed in: `b169b5f472116c50214037fbcfd8fa682ec45cbc`. The client provided the following explanation:

> See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/29

File(s) affected: `ConnectorRegistry.sol, VaultUpgradeableBeacon.sol, Vault.sol`

Description: At three locations in the code, `view` functions are used and may revert in specific conditions:

1. The function `ConnectorRegistry.get()` returns the current address associated with a connector name, but reverts if the connector name is paused or no address is associated with it;
2. The function `VaultUpgradeableBeacon.implementation()` returns the current `_implementation` address of the contract, but reverts if the contract is paused;
3. The function `Vault.totalAssets()` is used by several other functions and that returns the current amount of assets deposited in the integrated protocol, due to a call to the function `ConnectorRegistry.get()`. According to the EIP-4626, the function `totalAssets()` must not revert.

As a result, this potential to revert could disrupt the operations of external observers and make the contract not `EIP-4626` compliant.

Recommendation: Consider using distinct `view` functions based on the use case. For instance, the two functions listed above can be renamed `getOrRevert()`, and an alternative function that never reverts could be created, to be used by external observers.

KIDF-8 Missing Input Validation

Low Fixed

Update

The client addressed the recommended item.

Update

Marked as "Fixed" by the client. Addressed in: `c91c3e67ab0317c8691bb668945e5ab98912bd19`. The client provided the following explanation:

> We added the check for the shares/assets amount.

File(s) affected: `ConnectorRegistry.sol, FeeDispatcher.sol, Vault.sol`

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error:

In `Vault`:
1. `assets` and `shares` values can have the value `0`.

Recommendation: We recommend adding the relevant checks.

KIDF-9
The Function `_dispatchFees()` Is Exposed to Denial of Service

Low | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> A list that is too large should never happen. It's managed by the "FEE_MANAGER".

File(s) affected: `FeeDispatcher.sol`

Description: The function `_dispatchFees()` is used to dispatch the fees to all fee recipients. It can fail because of at least two scenarios:
1. the list of fee recipients is too large. In that case, the for loop will revert due to an out-of-gas error;
2. the token used for the fees has a blacklist mechanism, and one of the fee recipients is blacklisted;

If one of these scenarios happens, the issue can be solved since the current codebase lets the `FEE_MANAGER_ROLE` update the list of fee recipients.

Recommendation: Consider:
1. enforcing a max limit for the number of items in the list of fee recipients;
2. updating the logic to dispatch the fees by decoupling the actions to dispatch fees (executable by anyone) and to claim fees (executable by the beneficiaries). This would be supported by a storage mapping `claimableAmountByFeeRecipient`.

KIDF-10
Pending Fees Are Not Dispatched when the List of Fee Recipients Is Updated

Low | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> We want to keep it if a fee recipient's recipient private key has been lost. We need that flexibility.

File(s) affected: `FeeDispatcher.sol`

Description: Based on deposits and withdrawal operations, the vault accumulates performance and management fees to be dispatched to fee recipients. However, if the list of fee recipients is updated, pending fees that accumulated in the contract but were not dispatched yet will not be dispatched to the former recipients, in the former proportions.

Recommendation: Consider dispatching fees when the function `_setFeeRecipients()` is executed. Note that:
- checking if the accumulated fees are `0` may not work since dust can accumulate in the contract and not be claimable;
- the current list of fee recipients may lead to an error when fees are dispatched. For this reason, a try/catch structure may be relevant to prevent locking as well the execution of the function `_setFeeRecipients()`;

KIDF-11 Upgradability

Low | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> We need to keep the contract upgradeable. However, the functions to freeze the beacon implementation and the connectors are available.

File(s) affected: `Vault.sol`

Description: While upgradability is not a vulnerability in itself, users should be aware that the contract `Vault.sol` can be upgraded at any given time. This audit does not guarantee the behavior of future updates of this contract.

Recommendation: The fact that the contract can be upgraded and reasons for future upgrades should be communicated to users beforehand.

KIDF-12
Integration Requirements for Smart Contracts Are Not Documented

• Low Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > Everything is already documented in the Natspec documentation and in the ERC-4626 specification. We won't add anything else; this is enough.

File(s) affected: `Vault.sol`

Description: When an address performs a deposit, tokens are transferred to the Vault by executing the functions `mint()` and `deposit()`. Then, it may want to withdraw these tokens by executing the functions `redeem()` and `withdraw()`; Also, if the parameter `receiver` is incorrect or not able to withdraw, the associated amount will be lost;

If the address is an EOA, it can trigger all these functions. However, if it is a smart contract, it should be able to interact with these functions, with the correct values. Right now, that aspect is not documented.

Recommendation: Consider documenting integration requirements for smart contracts.

KIDF-13
Ambiguous Events when Fees Are Collected and Dispatched

• Low Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > We do not find it ambiguous from our point of view.

File(s) affected: `FeeDispatcher.sol`

Description: External observers can use events to calculate the current amount of management and performance fees that have been collected, dispatched, or are still available in the contract. However, the events `PerformanceFeesCollected` and `ManagementFeesCollected` only log the amount that has been collected during one operation. As a result, external observers can only deduce the amount of fees in the contract at a given time by observing all events, instead of only one event.

Recommendation: Consider adding a field `uint256 amountAfterOperation` to the events `PerformanceFeesCollected` and `ManagementFeesCollected`.

KIDF-14
User May Not Be Able to Deposit Tokens with Non-Standard Decimals

• Low Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > We will only deploy Vaults for tokens that meet the ERC-20 standard.

File(s) affected: `Vault.sol`

Description: Looking at the function `_previewMint()`, we see the following lines

```
uint256 _scaledRawAssetValue = _rawAssetValue * 10 ** _decimals;
uint256 _adjustedMaxPercent = (_MAX_PERCENT * 10 ** _decimals) - _managementFee;
uint256 _factor = _scaledRawAssetValue / _adjustedMaxPercent;
managementFeeAmount = _factor.mulDiv(_managementFee, 10 ** _decimals);
assets = _factor * _MAX_PERCENT;
```

If the variable `_factor` is rounded down to zero, `assets` will be zero, meaning the user will not be able to deposit into the vault. This happens when `rawAssetValue < (100 - managementPercent)`. In the worst case, `managementPercent = 0`, meaning `rawAssetValue` has to be at least 100 for the deposit to go through. To give a concrete example, `WBTC` uses `8` decimal places, and assuming the current price of around `50000` as well as `0` management fee, we need at least `~0.05 USD` worth of `WBTC` for the deposit to work. This is unlikely to be an issue for standard ERC20 tokens using 18 decimal places, but we simply want to raise awareness of this in the event there is a token that uses non-standard decimals.

Recommendation: Consider whether this is acceptable and document this potential limitation if necessary.

KIDF-15
Function `dispatchFees()` Is Not Protected Against Reentrancies

Low | Fixed

Update
The client incorporated a `nonReentrant` protection.

Update
Marked as "Fixed" by the client. Addressed in: `11073f6be807b3126f1c5e093d838463f6860a18`. The client provided the following explanation:

> See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/31

File(s) affected: `Vault.sol`

Description: The `dispatchFees()` function is currently accessible by any user, allowing them to distribute fees to predefined recipients. This setup is a breach of the principle of least privilege, which dictates that only fee recipients should have the authority to call this function. Moreover, the lack of `nonReentrant` protection for `dispatchFees()` poses a significant risk, as it does not prevent the function from being re-entered. An issue could happen if a callback mechanism can be triggered when the token is sent (`asset.safeTransfer()`). The CEI pattern is not respected in this function, because the storage variables `$._pendingManagementFee` and `$._pendingPerformanceFee` are updated at the end. From the callback, we can call any function manipulating these variables, for instance the `deposit()` that involves a call to `_incrementPendingManagementFee()` on a stale state of `$._pendingManagementFee`. In this specific case, a reentrancy of the contract could break its accounting.

Recommendation: Incorporate `nonReentrant` protection to ensure that, although the function can be invoked by any user, it prevents re-entering the contract to manipulate other sensitive functions.

KIDF-16
Risks of Supporting Non-Standard Erc-20 Tokens

Informational | Acknowledged

Update
Marked as "Acknowledged" by the client. The client provided the following explanation:

> We will only deploy Vaults for tokens that meet the ERC-20 standard.

File(s) affected: `Vault.sol`

Description: Supporting tokens with specific features such as fees, rebasing, pausable, upgradeable, blacklist-able, or hooks on transfers could negatively impact the main flows of the system if no specific mitigation measure is enforced to limit the consequences.

Recommendation: Consider analyzing thoroughly from a security perspective any token you would like to be supported.

KIDF-17

Complexity of the Function `_setFeeRecipients()` Can Be Reduced to Save Gas

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > Although it could be reduced, we will keep this function as is since it is not called frequently.

File(s) affected: `FeeDispatcher.sol`

Description: A new list of `FeeRecipient` is passed to the function `_setFeeRecipients()`, which ensures in nested for loops that the list does not contain duplicated items. The complexity of this check could be reduced by requiring the list of `FeeRecipient` to contain sorted items, ultimately saving gas. Then, the function can only use one loop to check that each item is strictly greater than the previous item and reverts if it is not the case. Another option could be to use a different data structure, such as `EnumerableSet`, for this purpose.

Recommendation: Consider refactoring the function to remove the nested for loop.

KIDF-18

Users Cannot Withdraw Same Amount of Assets Due to Precision Loss

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > This issue is from the ERC-4626 standard design; we won't move away from the standard.

File(s) affected: `Vault.sol`

Description: In `Vault.sol`, the `deposit()` and `withdraw()` functions are used to deposit and withdraw assets from the vault. Both functions rely on `_convertToShares()` to calculate the assets-share conversion. However, `deposit` and `withdraw` uses different rounding methods: `deposit` uses flooring, while `withdraw` uses ceiling. This could cause precision loss. As a result, users may not be able to withdraw the same amount of assets that they deposited. As according to the definition of the Vault, users shall not lose any assets other than the management fee, so this is not ideal.

Exploit Scenario: Consider a scenario with no fee in the protocol.

1. User `A` deposits 51 assets and gets 51 shares.
2. 50 rewards are added. Then `A` withdraws 1 asset with 1 share, leaving the total assets at 100 and the total of shares at 50.
3. User `B` deposits 101 assets and gets 50 shares due to flooring.
4. User `B` now tries to withdraw 101 assets. However, the ceiling rounding will cause the user to withdraw a maximum of 100 assets.

Recommendation: We recommend the team to consider if this is the intended behavior. Document the potential precision loss in the documentation and inform the users about it.

KIDF-19

Management Fee Can Be Bypassed Due to Precision Loss

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > Adding management fees will not solve issues, even if the cost is minimal. We must focus on identifying and addressing the root cause of problems instead of resorting to ineffective quick-fixes.

File(s) affected: `Vault.sol`

Description: In `Vault.sol`, the management fee is calculated by `mulDiv`:

```
managementFeeAmount = assets.mulDiv($._managementFee, _MAX_PERCENT * 10 **
IERC20Metadata(asset()).decimals());
```

Therefore, if the calculation result is less than 1, the `managementFeeAmount` will be returned as `0`. This could potentially cause issues as an adversary may choose to deposit a minimum amount multiple times to avoid paying for the management fee. Note that this is only profitable for the adversary if the management fee is higher than the gas fee for multiple deposits. Still, the Vault may not receive the management fee as expected if the attacker chooses to do so.

Note: it seems that the team is aware of this, but did not implement any additional features to prevent it. In test file `Vault.t.sol`, there is one line of `uint256 depositAmount = 1; // avoid management fee with 1`.

Recommendation: Confirm with the development team to check if this is the preferred operation. If not, set the minimum management fee to 1. This should not cause issues in most of the cases.

KIDF-20

Supply of Native Token May Not Be Supported for Compound V2 Connector

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > Our vault does not support native tokens.

File(s) affected: `CompoundV2Connector.sol`

Description: In Compound V2, there are two types of CTokens: `CEther` and `CErc20`. To mint `CEther` tokens, we call the function `function mint() payable`. However, this function signature is not supported by the function `CompoundV2Connector.deposit()`. The `deposit()` calls `cToken.mint(amount)`, which implies `cTokens` is of the type `CErc20` only, not taking into account of `CEther`.

Recommendation: Consider whether this is intended and introduce new features to take `CEther` into account if necessary.

KIDF-21

Salt Generation Mechanism to Create Vaults Could Be Tricked

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > The `createVault()` function is limited to the deployer and it is not an issue from our end.

File(s) affected: `VaultFactory.sol`

Description: In `VaultFactory.sol`, the `createVault()` function is used to create a new vault. This relies on the `salt` value to create the vault through `Create2`. However, such implementation requires a unique `salt` to be selected for each deployment. If the `salt` value mechanism can be predicted, an adversary may be able to trigger the deployment of specific vaults in advance. Depending on this mechanism's implementation, it may even result in the impossibility of a given address to create a vault. Another scenario could be the impossibility for a specific address to create more than vault if there is a 1:1 cardinality between an address and its derived `salt` value.

Recommendation: We recommend the team design a salt generation mechanism that cannot be tricked.

KIDF-22

Redundant Operations when Interacting with Morpho

Informational | Acknowledged

Update

Marked as "Acknowledged" by the client. The client provided the following explanation:

> We would like to maintain the same implementation as Morpho's implementation ⇒ https://github.com/morpho-org/morpho-v1-optimizers-vaults

File(s) affected: `MorphoAaveV2Connector.sol, MorphoCompoundV2Connector.sol`

Description: The functions `updateIndexes()` and `updateP2PIndexes()` are manually executed in the dedicated connectors. However, these functions are also called internally by the integrated protocol when the functions `supply()` and `withdraw()` are executed. These calls can be considered as an extra precaution to cover the case where the codebase of Morpho is updated and no more internal call is performed. However, they result in more gas costs.

Recommendation: Consider assessing if these operations should still be performed manually or not.

KIDF-23 Consolidation of Best Practices

Informational | Fixed

Update

The client addressed the items.

Update

Marked as "Fixed" by the client. Addressed in: `de8559c89b61912c37463c26e6e91fa9362c7fcf`, `39ba22ba74d22c5375e93f0ec0fbe8abd81e1f93`. The client provided the following explanation:

> See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/28

File(s) affected: `VaultFactory.sol, ConnectorRegistry.sol, FeeDispatcher.sol, Vault.sol`

Description: Here is a list of consolidated suggestions to improve the clarity, maintainability and gas efficiency of the codebase:
In `VaultFactory.sol`:

1. Consider if it is better for the function `createVault()` to return an `address` or a `VaultBeaconProxy` type.

In `ConnectorRegistry.sol`:

1. The fact that a given connector name is frozen is not logged.
2. The modifier `exists()` checks for the condition `if (_connectors[name] == address(0))`. This can be replaced with `if (!connectorExists(name))` to make use of the existing function.
3. The function `add()` checks for `if (_connectors[name] != address(0))`, which can be replaced with `if (connectorExists(name))`.
4. The function `unpaused()` checks for `if (pauseTimestamp[name] <= block.timestamp)`, which can be replaced with `if (!paused())`.

In `FeeDispatcher.sol`:

1. Consider caching in the function `feeRecipient()` the value of `$._feeRecipients.length` in a local variable to save gas;
2. Consider in the function `_dispatchFees()` declaring `currentRecipient` outside of the loop to prevent reassigning it at each loop, and save gas;

In `Vault.sol`:

1. Consider making it explicit that the `mulDiv()` operation in the function `_currentPerformanceFeeAmount()` should round down the result;

Recommendation: Consider addressing these items.

KIDF-24 If `totalAssets()` Decreases, the Operations `deposit()`, `mint()`, `withdraw()` and `redeem()` Will Revert

Undetermined | Mitigated

Update

The potential underflow in the function `_currentPerformanceFeeAmount()` is now managed and, in this scenario, the rewards will be `0`. Also, two additional functions were added `_totalAssetsForIncome()` and `_totalAssetsForOutcome()` to handle respectively the cases where an amount of tokens is deposited to the vault and withdrawn from the vault. These functions avoid a potential underflow when the total of performance fees is higher than the total assets, which can only happen depending on the amount of `collectablePerformanceFees`, because `_currentPerformanceFeeAmount()` would return `0` in this case.

We further suggest emitting an event as an alert to highlight the fact that we entered in the block `if ($._lastTotalAssets > _totalAssets) { return 0; }`.

Update

Marked as "Mitigated" by the client. Addressed in: `54c4e54a7ebb47fe9a1d6e70aa29aa0719ad5c85`, `19512953874160145524 59ab72cf5a5a8edbd6e8`, `5843733f3b67331b77ac977b91a1398e24256e0c`, `823af26fcbbaf443b146af3700413f5f57b5921f`. The client provided the following explanation:

> See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/24 (#Handle underlying protocol loss)

File(s) affected: `Vault.sol`

Description: In the current code logic, the `totalAssets()` function is used to calculate the total assets of the pool. Multiple key operations rely on the value returned by `totalAssets()`. However, the current design is based on the assumption that the `totalAssets()` value will only increase. If this assumption breaks, it would cause the operations `deposit()/mint()/withdraw()/redeem()` to revert due to the current implementation of how performance fees are collected.

Recommendation: We recommend avoiding the collection of `performanceFee` in `withdraw()` and `redeem()` functions when `totalAssets()` decrease. This strategy ensures users can still withdraw their funds in the event of a hack in the integrated DeFi protocol. Conversely, we advise against modifying the current method of collecting `performanceFee` in the `deposit()` and `mint()` functions. The rationale of leaving these two functions unfixed is that the current implementation prevents read-only reentrancy attacks by reducing the `totalAssets()` value to gain more shares than expected.

KIDF-25
Read-only Reentrancy Can Happen by Manipulating `totalAssets()`

Undetermined | Acknowledged

Update

Marked as "Acknowledged" by the client. The client provided the following explanation:

> We need to be careful when we introduce new protocols. It's important to make sure that the `totalAssets()` function doesn't decrease due to any manipulation.

File(s) affected: `Vault.sol`

Description: Read-only reentrancy could occur if a method is found to alter the return value of the `totalAsset()` function in any of the integrated DeFi protocols. This manipulation could potentially be achieved through various means. Here we provide a theoretical exploit scenario as an example.

Exploit Scenario:

1. The attacker initiates the exploit by taking a flash loan from a DeFi pool integrated with the vault.
2. This action results in a decrease in `_totalAsset()` due to the reduction of underlying assets in the DeFi pool. The exact impact on `_totalAsset()` may vary depending on the specific implementation of the DeFi protocols involved.
3. Subsequently, the attacker calls the `deposit()` function of the vault to mint shares.
4. Due to the decreased `_totalAsset()` value from step 2, the number of shares minted during the deposit in step 3 will be disproportionately high compared to the actual amount of assets added. Essentially, the reduction in `_totalAsset()` artificially inflates the value of the minted shares.

Recommendation: Incorporating an internal state variable for `_totalAsset()` to guarantee its value never decreases as an invariant during calls to `deposit()` or `mint()` will also bolster system security. Notably, the existing `performanceFee` collection mechanism already safeguards this invariant.
Moreover, continuous monitoring and auditing of the updated contracts for all integrated protocols (updates of the codebase or core on-chain protocol parameters) are critical to mitigating similar risks.

KIDF-26

`totalAssets()` May Provide an Outdated Amount of Assets for `CompoundV2` Connector

• Undetermined Fixed

> **Update**
> The client is now using a view function to get the current amount of assets.

> **Update**
> Marked as "Fixed" by the client. Addressed in: `9a8600b09dde27622d669c0957f24f61e2a8e414`. The client provided the following explanation:
>
> > See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/32

File(s) affected: `CompoundV2Connector.sol`

Description: In `CompoundV2Connector.totalAssets()`, we retrieve the total underlying assets with the following line

```
(, uint256 balance,, uint256 exchangeRate) = cToken.getAccountSnapshot(msg.sender);
```

However, per Compound's documentation, the `exchangeRate` is a cached value and may not be the most up-to-date, since it requires a state update (call to `accrueInterest()`) that cannot be made within a `view` function call. There is a function `balanceOfUnderlying()` (see here) that is more suited for the task of checking the most up-to-date balance of the underlying assets. However, this function is not a view function so it cannot be called from a view function.

Recommendation: Consider assessing carefully which function should be called in which context (view VS non-view functions).

KIDF-27

Possibility to repeatedly perform opposite actions at almost no cost

• Undetermined Acknowledged

> **Update**
> Marked as "Acknowledged" by the client. The client provided the following explanation:
>
> > Not an issue on our end.

File(s) affected: `Vault.sol`

Description: Integrators should be aware that setting a low `managementFee` makes it possible for users to repeatedly perform opposite actions (deposit/mint VS withdraw/redeem) within the same transaction at almost no cost. This does not make a lot of sense for legitimate users but, on the opposite, it could become the basis of malicious actions. Even if the audit team did not find any concrete attack for this scenario, the manipulation of the exchange rate of the integrated protocol (between deposited assets and shares owned by the vault) could make such an attack profitable.

Recommendation: Consider if an address should be able to perform more than one opposite operation (deposit/mint VS withdraw/redeem) per block or not. If not, update the code accordingly. Document to Integrators the fact that the `managementFee` can be used as a measure to reduce the profitability of such an attack.

KIDF-28

Addresses with the Role `PAUSER` Can Unpause Contracts Even if it Should Be Restricted to Addresses with the Role `UNPAUSER`

• Undetermined Fixed

> **Update**
> It is not possible anymore to pause the contracts with a value that will reduce the active pause duration.

Update

Marked as "Fixed" by the client. Addressed in: `f7cc0f7c92ca9d5ac045a5042edcd24a7eac1c57`. The client provided the following explanation:

> See the following PR ⇒ https://github.com/kilnfi/defi-integrations/pull/32

File(s) affected: `ContractRegistry.sol, VaultUpgradeableBeacon.sol`

Description: There are two distinct roles in the contracts `ContractRegistry.sol` and `VaultUpgradeableBeacon.sol` to pause and unpause the contracts, respectively via the functions `pause()`, `pauseFor()` and `unpause()`. Only addresses with the role `UNPAUSER` should be able to call the function `unpause()`. However, addresses with the role `PAUSER` can call the function `pauseFor()` with the value `1` and this will have approximately the same effect as unpausing the contract since it will not be paused anymore one second after the transaction is executed.

Recommendation: Consider merging the roles `PAUSER` and `UNPAUSER`, or enforcing at the contract level a minimum duration a contract can be paused for.

KIDF-29

Integrators Can Remove Kiln From the Fee Recipients

- Undetermined
- Acknowledged

Update

Marked as "Acknowledged" by the client. The client provided the following explanation:

> We manage it on our side through agreements with the integrators."

File(s) affected: `Vault.sol, FeeDispatcher.sol`

Description: Integrators own the role `FEE_MANAGER_ROLE`, which lets them execute the `setFeeRecipients()` to update the list of fee recipients and the underlying proportions. Since Kiln expects to benefit from the system by being a fee recipient, Kiln should be aware of the fact that it can be removed from the fee recipients by integrators.

Recommendation: Consider enforcing the fact that Kiln should always receive a given amount of the fees or acknowledging that it can be removed from the fee recipients by integrators.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not post an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Code Documentation

In `FeeDispatcher.sol`:

1. Replace `100e18` by `100e[underlyingDecimal]` in the documentation of the function `_setFeeRecipients()`;
2. For the events `PerformanceFeesCollected` and `ManagementFeesCollected`, consider describing if the values used represent the total amount collected or the amount that was collected during this operation.

In `Vault.sol`:

1. There is a mismatch between the comments and the associated line of code in the function `_deposit()` where the comments involve management fees but the code involves performance fees.

In `ConnectoryRegistry.sol`:

1. The modifier `whenNotPaused()` has the comment `Throws if the connector is frozen.`, but it should be `if the connector is paused`.

General observations:

1. The documentation does not include necessary introduction about how the management fee and performance fee are calculated and charged. It is recommended to add a brief introduction about the calculation of the management fee and performance fee in the documentation with formula, similar to the share/asset calculation.

Toolset

The notes below outline the setup and steps performed in the process of this audit.

Setup

Tool Setup:

- Slither

Steps taken to run the tools:

1. Install the Slither tool: `pip3 install slither-analyzer`
2. Run Slither from the project directory: `slither .`

Automated Analysis

Slither

We analyzed the output of Slither and true positive items were added to this report.

Test Suite Results

All tests pass. The test suite consists of unit tests (all contracts except the connectors) and end-to-end tests (only the connectors). Unhappy paths are not covered if integrated protocols stop working as expected.

Fix Review update

Tests were added.

```
Running 1 test for test/e2e/CompoundV3Connector.t.sol:CompoundV3ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 465568)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.26s

Running 1 test for test/e2e/AaveV2Connector.t.sol:AaveV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 806891)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.89s

Running 1 test for test/e2e/CompoundV2Connector.t.sol:CompoundV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 623003)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.14s

Running 1 test for test/e2e/AaveV3Connector.t.sol:AaveV3ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 708887)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.27s

Running 15 tests for test/FeeDispatcher.t.sol:FeeDispatcherTest
[PASS] test_cannotGetNonExistingRecipient() (gas: 23469)
[PASS] test_cannotInitializeAfterSetup() (gas: 26972)
```

```
[PASS] test_cannotSetEmptyFeeRecipient() (gas: 9219)
[PASS] test_cannotSetFeeRecipientWithDuplicateRecipient() (gas: 38576)
[PASS] test_cannotSetFeeRecipientWithZeroAddress() (gas: 32735)
[PASS] test_cannotSetFeeRecipientWithZeroManagementFeeSplit() (gas: 38706)
[PASS] test_cannotSetFeeRecipientWithZeroPerformanceFeeSplit() (gas: 39026)
[PASS] test_cannotSetFeeRecipientsWithTotalFeeSplitGreaterThan100Percent() (gas: 85366)
[PASS] test_cannotSetFeeRecipientsWithTotalFeeSplitLessThan100Percent() (gas: 79702)
[PASS] test_fuzz_PendingManagementFee(uint256) (runs: 256, μ: 29358, ~: 29980)
[PASS] test_fuzz_PendingPerformanceFee(uint256) (runs: 256, μ: 29102, ~: 30035)
[PASS] test_fuzz_dispatchFees(uint128,uint128) (runs: 256, μ: 184439, ~: 188371)
[PASS] test_initial_FeeRecipientAt() (gas: 24174)
[PASS] test_initial_FeeRecipients() (gas: 31319)
[PASS] test_initialize() (gas: 1276009)
Test result: ok. 15 passed; 0 failed; 0 skipped; finished in 4.98s

Running 4 tests for test/VaultFactory.t.sol:VaultFactoryTest
[PASS] test_cannot_createVaultIfNotDeployer(address) (runs: 256, μ: 42064, ~: 42064)
[PASS] test_cannot_deployIfInvalidBeacon(address) (runs: 256, μ: 115063, ~: 115063)
[PASS] test_cannot_deployIfInvalidConnectorRegistry(address) (runs: 256, μ: 117759, ~: 117769)
[PASS] test_createVault() (gas: 741305)
Test result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.23s

Running 20 tests for test/VaultUpgradeableBeacon.t.sol:VaultUpgradeableBeaconTest
[PASS] test_cannotUpdateImplementationIfFrozen() (gas: 36448)
[PASS] test_cannot_freezeIfAlreadyFrozen() (gas: 38497)
[PASS] test_cannot_freezeIfNotFreezer(address) (runs: 256, μ: 16263, ~: 16263)
[PASS] test_cannot_getImplementationIfPaused() (gas: 39174)
[PASS] test_cannot_pauseForIfNotPauser(address) (runs: 256, μ: 16245, ~: 16245)
[PASS] test_cannot_pauseForIfZeroDuration() (gas: 14613)
[PASS] test_cannot_pauseIfNotPauser(address) (runs: 256, μ: 16196, ~: 16196)
[PASS] test_cannot_unPauseIfNotPaused() (gas: 16362)
[PASS] test_cannot_unpauseIfNotUnpauser(address) (runs: 256, μ: 44015, ~: 44015)
[PASS] test_cannot_upgradeIfNotImplementationManager(address) (runs: 256, μ: 3840179, ~: 3840179)
[PASS] test_cannot_upgradeInvalidImplementation(address) (runs: 256, μ: 20426, ~: 20436)
[PASS] test_deploy() (gas: 35791)
[PASS] test_freeze() (gas: 38102)
[PASS] test_pause() (gas: 38202)
[PASS] test_pauseFor(uint256) (runs: 256, μ: 42396, ~: 42396)
[PASS] test_pauseForAndUnPause(uint256) (runs: 256, μ: 33455, ~: 33475)
[PASS] test_pauseForTwice(uint256,uint256) (runs: 256, μ: 48217, ~: 48217)
[PASS] test_unPause(uint256) (runs: 256, μ: 32477, ~: 32494)
[PASS] test_upgrade() (gas: 3847496)
[PASS] test_upgradeToSame() (gas: 27737)
Test result: ok. 20 passed; 0 failed; 0 skipped; finished in 16.07s

Running 1 test for test/e2e/MorphoCompoundV2Connector.t.sol:MorphoCompoundV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 1729174)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 29.99s

Running 1 test for test/e2e/SDAIConnector.t.sol:SDAIConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 572305)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.40s

Running 1 test for test/e2e/MorphoAaveV2Connector.t.sol:MorphoAaveV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 1702152)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 46.24s

Running 31 tests for test/ConnectorRegistry.t.sol:ConnectorRegistryTest
[PASS] test_add(uint8,bytes32) (runs: 256, μ: 413793, ~: 413793)
[PASS] test_cannot_addIfAlreadyExists() (gas: 408033)
[PASS] test_cannot_addIfNotConnectorManager(address,bytes32) (runs: 256, μ: 383691, ~: 383691)
[PASS] test_cannot_addIfNotContract() (gas: 26491)
[PASS] test_cannot_freezeIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 409889, ~: 409889)
[PASS] test_cannot_freezeIfFrozen(bytes32) (runs: 256, μ: 433376, ~: 433376)
[PASS] test_cannot_freezeIfNotFreezer(address,bytes32) (runs: 256, μ: 414918, ~: 414918)
[PASS] test_cannot_pauseForIfDoesNotExist(bytes32) (runs: 256, μ: 409951, ~: 409951)
[PASS] test_cannot_pauseForIfNotPauser(address,bytes32) (runs: 256, μ: 412624, ~: 412624)
[PASS] test_cannot_pauseForIfTimestampZero(bytes32) (runs: 256, μ: 409618, ~: 409618)
```

```
[PASS] test_cannot_pauseIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 409955, ~: 409955)
[PASS] test_cannot_pauseIfNotPauser(address,bytes32) (runs: 256, μ: 412651, ~: 412651)
[PASS] test_cannot_removeIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 409868, ~: 409868)
[PASS] test_cannot_removeIfFrozen(bytes32) (runs: 256, μ: 433455, ~: 433455)
[PASS] test_cannot_removeIfNotConnectorManager(address,bytes32) (runs: 256, μ: 414853, ~: 414853)
[PASS] test_cannot_unPauseIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 409869, ~: 409869)
[PASS] test_cannot_unPauseIfNotPaused(bytes32) (runs: 256, μ: 412086, ~: 412086)
[PASS] test_cannot_unPauseIfNotUnpauser(address,bytes32) (runs: 256, μ: 412717, ~: 412717)
[PASS] test_cannot_updateIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 410021, ~: 410021)
[PASS] test_cannot_updateIfFrozen(bytes32) (runs: 256, μ: 433661, ~: 433661)
[PASS] test_cannot_updateIfNotConnectorManager(address,bytes32) (runs: 256, μ: 782170, ~: 782170)
[PASS] test_cannot_updateIfNotContract(bytes32) (runs: 256, μ: 419475, ~: 419475)
[PASS] test_deploy() (gas: 10297)
[PASS] test_freeze(bytes32) (runs: 256, μ: 432643, ~: 432643)
[PASS] test_pause(bytes32) (runs: 256, μ: 433966, ~: 433966)
[PASS] test_pauseFor(uint256) (runs: 256, μ: 436205, ~: 436205)
[PASS] test_pauseForAndUnPause(uint256) (runs: 256, μ: 423854, ~: 423854)
[PASS] test_pauseForTwice(uint256,uint256) (runs: 256, μ: 429363, ~: 429363)
[PASS] test_remove(bytes32) (runs: 256, μ: 395603, ~: 395603)
[PASS] test_unPause(bytes32) (runs: 256, μ: 420284, ~: 420284)
[PASS] test_update(bytes32) (runs: 256, μ: 781885, ~: 781885)
Test result: ok. 31 passed; 0 failed; 0 skipped; finished in 46.72s

Running 43 tests for test/Vault.t.sol:VaultTest
[PASS] test_approveAndTransferFromIfTransferable() (gas: 413645)
[PASS] test_cannot_approveIfNotTransferable() (gas: 247132)
[PASS] test_cannot_collectFeesIfNoRewards() (gas: 61226)
[PASS] test_cannot_collectPerformanceFeesIfNotOwner(address) (runs: 256, μ: 33008, ~: 33008)
[PASS] test_cannot_depositIfConnectorPaused() (gas: 85329)
[PASS] test_cannot_depositMoreThanMaxDeposit(uint256) (runs: 256, μ: 114365, ~: 114365)
[PASS] test_cannot_initializeNotDelegateCall() (gas: 3852624)
[PASS] test_cannot_initializeWrongConnectorName() (gas: 4872526)
[PASS] test_cannot_initializeWrongConnectorRegistry() (gas: 4849845)
[PASS] test_cannot_initializeWrongManagementFee() (gas: 4820302)
[PASS] test_cannot_initializeWrongPerformanceFee() (gas: 4795805)
[PASS] test_cannot_mintIfConnectorPaused() (gas: 80169)
[PASS] test_cannot_mintMoreThanMaxMint(uint256) (runs: 256, μ: 114429, ~: 114429)
[PASS] test_cannot_redeemIfConnectorPaused() (gas: 315602)
[PASS] test_cannot_redeemMoreThanMaxRedeem(uint256) (runs: 256, μ: 287429, ~: 287429)
[PASS] test_cannot_setManagementFeeIfNotOwner(address) (runs: 256, μ: 27955, ~: 27955)
[PASS] test_cannot_setPerformanceFeeIfNotOwner(address) (runs: 256, μ: 27881, ~: 27881)
[PASS] test_cannot_setRecipientsIfNotOwner(address) (runs: 256, μ: 44203, ~: 44203)
[PASS] test_cannot_setWrongManagementFee(uint256) (runs: 256, μ: 31584, ~: 31584)
[PASS] test_cannot_setWrongPerformanceFee(uint256) (runs: 256, μ: 58303, ~: 58303)
[PASS] test_cannot_transferFromIfNotTransferable() (gas: 247332)
[PASS] test_cannot_transferIfNotTransferable() (gas: 247199)
[PASS] test_cannot_withdrawIfConnectorPaused() (gas: 305077)
[PASS] test_cannot_withdrawMoreThanMaxWithdraw(uint256) (runs: 256, μ: 290886, ~: 290886)
[PASS] test_collectPerformanceFeeWhenDepositing() (gas: 491338)
[PASS] test_collectPerformanceFees(uint256) (runs: 256, μ: 373421, ~: 385030)
[PASS] test_collectPerformanceFeesAfterRemovingFees(uint256) (runs: 256, μ: 398509, ~: 404849)
[PASS] test_depositAndWithdrawFuzzOffset(uint8,uint256) (runs: 256, μ: 929154, ~: 932186)
[PASS] test_deposit_rewards_withdraw_dispatch() (gas: 449832)
[PASS] test_initialize(uint256,uint256,bool,string,string) (runs: 256, μ: 5014166, ~: 5008487)
[PASS] test_initializeSetUp() (gas: 44879)
[PASS] test_mint_rewards_redeem_dispatch() (gas: 475489)
[PASS] test_previewDeposit_first(uint256) (runs: 256, μ: 54598, ~: 54598)
[PASS] test_previewMint_first(uint256) (runs: 256, μ: 55687, ~: 55687)
[PASS] test_previewRedeem_first(uint256) (runs: 256, μ: 52603, ~: 52603)
[PASS] test_previewWithdraw_first(uint256) (runs: 256, μ: 52669, ~: 52669)
[PASS] test_preview_secondAndRewards() (gas: 307983)
[PASS] test_redeemWithAllowance() (gas: 435460)
[PASS] test_removeFees() (gas: 445056)
[PASS] test_setFeeRecipients() (gas: 69309)
[PASS] test_setManagementFee(uint256) (runs: 256, μ: 38753, ~: 39072)
[PASS] test_setPerformanceFee(uint256) (runs: 256, μ: 63483, ~: 63708)
[PASS] test_transferIfTransferable() (gas: 390430)
Test result: ok. 43 passed; 0 failed; 0 skipped; finished in 40.98s
```

```
Ran 12 test suites: 120 tests passed, 0 failed, 0 skipped (120 total tests)

**Fix Review update**

Running 15 tests for test/FeeDispatcher.t.sol:FeeDispatcherTest
[PASS] test_cannotGetNonExistingRecipient() (gas: 23291)
[PASS] test_cannotInitializeAfterSetup() (gas: 26972)
[PASS] test_cannotSetEmptyFeeRecipient() (gas: 9241)
[PASS] test_cannotSetFeeRecipientWithDuplicateRecipient() (gas: 38576)
[PASS] test_cannotSetFeeRecipientWithZeroAddress() (gas: 32735)
[PASS] test_cannotSetFeeRecipientWithZeroManagementFeeSplit() (gas: 38706)
[PASS] test_cannotSetFeeRecipientWithZeroPerformanceFeeSplit() (gas: 39044)
[PASS] test_cannotSetFeeRecipientsWithTotalFeeSplitGreaterThan100Percent() (gas: 85344)
[PASS] test_cannotSetFeeRecipientsWithTotalFeeSplitLessThan100Percent() (gas: 79724)
[PASS] test_fuzz_PendingManagementFee(uint256) (runs: 256, μ: 29425, ~: 30047)
[PASS] test_fuzz_PendingPerformanceFee(uint256) (runs: 256, μ: 28635, ~: 30035)
[PASS] test_fuzz_dispatchFees(uint128,uint128) (runs: 256, μ: 184188, ~: 188462)
[PASS] test_initial_FeeRecipientAt() (gas: 24176)
[PASS] test_initial_FeeRecipients() (gas: 31263)
[PASS] test_initialize() (gas: 1277415)
Test result: ok. 15 passed; 0 failed; 0 skipped; finished in 53.48ms

Running 32 tests for test/ConnectorRegistry.t.sol:ConnectorRegistryTest
[PASS] test_add(uint8,bytes32) (runs: 256, μ: 544592, ~: 544592)
[PASS] test_cannot_addIfAlreadyExists() (gas: 541128)
[PASS] test_cannot_addIfNotConnectorManager(address,bytes32) (runs: 256, μ: 516737, ~: 516737)
[PASS] test_cannot_addIfNotContract() (gas: 26467)
[PASS] test_cannot_freezeIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 542972, ~: 542972)
[PASS] test_cannot_freezeIfFrozen(bytes32) (runs: 256, μ: 567636, ~: 567636)
[PASS] test_cannot_freezeIfNotFreezer(address,bytes32) (runs: 256, μ: 547845, ~: 547845)
[PASS] test_cannot_pauseForBelowCurrentPauseTimestamp(uint256,uint256) (runs: 256, μ: 574268, ~:
574457)
[PASS] test_cannot_pauseForIfDoesNotExist(bytes32) (runs: 256, μ: 542967, ~: 542967)
[PASS] test_cannot_pauseForIfNotPauser(address,bytes32) (runs: 256, μ: 545640, ~: 545640)
[PASS] test_cannot_pauseForIfTimestampZero(bytes32) (runs: 256, μ: 542734, ~: 542734)
[PASS] test_cannot_pauseIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 543015, ~: 543015)
[PASS] test_cannot_pauseIfNotPauser(address,bytes32) (runs: 256, μ: 545667, ~: 545667)
[PASS] test_cannot_removeIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 542950, ~: 542950)
[PASS] test_cannot_removeIfFrozen(bytes32) (runs: 256, μ: 567759, ~: 567759)
[PASS] test_cannot_removeIfNotConnectorManager(address,bytes32) (runs: 256, μ: 547824, ~: 547824)
[PASS] test_cannot_unPauseIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 542973, ~: 542973)
[PASS] test_cannot_unPauseIfNotPaused(bytes32) (runs: 256, μ: 545190, ~: 545190)
[PASS] test_cannot_unPauseIfNotUnpauser(address,bytes32) (runs: 256, μ: 545755, ~: 545755)
[PASS] test_cannot_updateIfConnectorDoesNotExist(bytes32) (runs: 256, μ: 543170, ~: 543170)
[PASS] test_cannot_updateIfFrozen(bytes32) (runs: 256, μ: 567877, ~: 567877)
[PASS] test_cannot_updateIfNotConnectorManager(address,bytes32) (runs: 256, μ: 1048197, ~:
1048197)
[PASS] test_cannot_updateIfNotContract(bytes32) (runs: 256, μ: 552580, ~: 552580)
[PASS] test_deploy() (gas: 10210)
[PASS] test_freeze(bytes32) (runs: 256, μ: 566860, ~: 566860)
[PASS] test_pause(bytes32) (runs: 256, μ: 566883, ~: 566883)
[PASS] test_pauseFor(uint256) (runs: 256, μ: 569392, ~: 569392)
[PASS] test_pauseForAndUnPause(uint256) (runs: 256, μ: 557052, ~: 557052)
[PASS] test_pauseForTwice(uint256,uint256) (runs: 256, μ: 563879, ~: 563780)
[PASS] test_remove(bytes32) (runs: 256, μ: 526496, ~: 526496)
[PASS] test_unPause(bytes32) (runs: 256, μ: 553277, ~: 553277)
[PASS] test_update(bytes32) (runs: 256, μ: 1045779, ~: 1045779)
Test result: ok. 32 passed; 0 failed; 0 skipped; finished in 559.94ms

Running 1 test for test/e2e/MorphoAaveV2Connector.t.sol:MorphoAaveV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 1755554)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 754.65ms

Running 1 test for test/e2e/MorphoCompoundV2Connector.t.sol:MorphoCompoundV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 1788650)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 756.58ms

Running 4 tests for test/VaultFactory.t.sol:VaultFactoryTest
```

```
[PASS] test_cannot_createVaultIfNotDeployer(address) (runs: 256, μ: 45198, ~: 45198)
[PASS] test_cannot_deployIfInvalidBeacon(address) (runs: 256, μ: 115119, ~: 115119)
[PASS] test_cannot_deployIfInvalidConnectorRegistry(address) (runs: 256, μ: 117826, ~: 117826)
[PASS] test_createVault() (gas: 872548)
Test result: ok. 4 passed; 0 failed; 0 skipped; finished in 60.47ms

Running 1 test for test/e2e/CompoundV3Connector.t.sol:CompoundV3ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 501126)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.00s

Running 1 test for test/e2e/SDAIConnector.t.sol:SDAIConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 605614)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.00s

Running 1 test for test/e2e/CompoundV2ConnectorDai.t.sol:CompoundV2ConnectorDaiE2E
[PASS] test_depositAndWithdraw() (gas: 679966)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 432.89ms

Running 1 test for test/e2e/AaveV3Connector.t.sol:AaveV3ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 754411)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.01s

Running 2 tests for test/e2e/SanctionsList.t.sol:SanctionsListE2E
[PASS] test_chainalysis_cannot_depositIfSanctioned() (gas: 78383)
[PASS] test_chainalysis_depositIfNotSanctioned() (gas: 515984)
Test result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.01s

Running 1 test for test/e2e/AaveV2Connector.t.sol:AaveV2ConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 852988)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.01s

Running 2 tests for test/e2e/CompoundV2Connector.t.sol:CompoundV2ConnectorE2E
[PASS] test_claimComp() (gas: 714295)
[PASS] test_depositAndWithdraw() (gas: 719667)
Test result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.02s

Running 1 test for test/e2e/MetamorphoConnector.t.sol:MetamorphoConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 1586999)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 951.15ms

Running 21 tests for test/VaultUpgradeableBeacon.t.sol:VaultUpgradeableBeaconTest
[PASS] test_cannotUpdateImplementationIfFrozen() (gas: 36247)
[PASS] test_cannot_freezeIfAlreadyFrozen() (gas: 38327)
[PASS] test_cannot_freezeIfNotFreezer(address) (runs: 256, μ: 16136, ~: 16136)
[PASS] test_cannot_getImplementationIfPaused() (gas: 39110)
[PASS] test_cannot_pauseForBelowCurrentPauseTimestamp(uint256,uint256) (runs: 256, μ: 45131, ~:
45276)
[PASS] test_cannot_pauseForIfNotPauser(address) (runs: 256, μ: 16184, ~: 16184)
[PASS] test_cannot_pauseForIfZeroDuration() (gas: 14499)
[PASS] test_cannot_pauseIfNotPauser(address) (runs: 256, μ: 16091, ~: 16091)
[PASS] test_cannot_unPauseIfNotPaused() (gas: 16281)
[PASS] test_cannot_unpauseIfNotUnpauser(address) (runs: 256, μ: 43817, ~: 43817)
[PASS] test_cannot_upgradeIfNotImplementationManager(address) (runs: 256, μ: 4683330, ~: 4683330)
[PASS] test_cannot_upgradeInvalidImplementation(address) (runs: 256, μ: 20332, ~: 20332)
[PASS] test_deploy() (gas: 35973)
[PASS] test_freeze() (gas: 38010)
[PASS] test_pause() (gas: 38132)
[PASS] test_pauseFor(uint256) (runs: 256, μ: 42687, ~: 42640)
[PASS] test_pauseForAndUnPause(uint256) (runs: 256, μ: 33743, ~: 33723)
[PASS] test_pauseForTwice(uint256,uint256) (runs: 256, μ: 49370, ~: 49233)
[PASS] test_unPause(uint256) (runs: 256, μ: 32739, ~: 32724)
[PASS] test_upgrade() (gas: 4690648)
[PASS] test_upgradeToSame() (gas: 27623)
Test result: ok. 21 passed; 0 failed; 0 skipped; finished in 204.67ms

Running 1 test for test/e2e/MorphoBlueConnector.t.sol:MorphoBlueConnectorE2E
[PASS] test_depositAndWithdraw() (gas: 567311)
Test result: ok. 1 passed; 0 failed; 0 skipped; finished in 458.87ms
```

```
Running 66 tests for test/Vault.t.sol:VaultTest
[PASS] test_approveAndTransferFromIfTransferable() (gas: 420940)
[PASS] test_cannot_approveIfNotTransferable() (gas: 251101)
[PASS] test_cannot_claimIfNotClaimManager(address) (runs: 256, μ: 35467, ~: 35467)
[PASS] test_cannot_claimIfNothingToClaim(uint256) (runs: 256, μ: 274513, ~: 274431)
[PASS] test_cannot_claimIfTotalAssetsDecreased(uint256,uint256) (runs: 256, μ: 279387, ~: 279428)
[PASS] test_cannot_collectFeesIfNoRewards() (gas: 59398)
[PASS] test_cannot_collectPerformanceFeesIfNotOwner(address) (runs: 256, μ: 32894, ~: 32894)
[PASS] test_cannot_depositIfAmountZero() (gas: 36375)
[PASS] test_cannot_depositIfConnectorPaused() (gas: 258490)
[PASS] test_cannot_depositIfDepositPaused(uint256) (runs: 256, μ: 131593, ~: 131425)
[PASS] test_cannot_depositIfSanctioned() (gas: 141445)
[PASS] test_cannot_depositMoreThanMaxDeposit(uint256) (runs: 256, μ: 123466, ~: 123678)
[PASS] test_cannot_initializeNotDelegateCall() (gas: 4695306)
[PASS] test_cannot_initializeWrongConnectorName() (gas: 5721915)
[PASS] test_cannot_initializeWrongConnectorRegistry() (gas: 5699241)
[PASS] test_cannot_initializeWrongManagementFee() (gas: 5669698)
[PASS] test_cannot_initializeWrongPerformanceFee() (gas: 5645119)
[PASS] test_cannot_mintIfAmountZero() (gas: 36399)
[PASS] test_cannot_mintIfConnectorPaused() (gas: 260224)
[PASS] test_cannot_mintMoreThanMaxMint(uint256) (runs: 256, μ: 123327, ~: 123521)
[PASS] test_cannot_pauseDeposit_ifNotPauser(address) (runs: 256, μ: 27822, ~: 27822)
[PASS] test_cannot_redeemIfAmountZero() (gas: 36297)
[PASS] test_cannot_redeemIfConnectorPaused() (gas: 335625)
[PASS] test_cannot_redeemMoreThanMaxRedeem(uint256) (runs: 256, μ: 298563, ~: 298469)
[PASS] test_cannot_setManagementFeeIfNotOwner(address) (runs: 256, μ: 27885, ~: 27885)
[PASS] test_cannot_setPerformanceFeeIfNotOwner(address) (runs: 256, μ: 27756, ~: 27756)
[PASS] test_cannot_setRecipientsIfNotOwner(address) (runs: 256, μ: 44169, ~: 44169)
[PASS] test_cannot_setSanctionsListIfNotSanctionsManager(address) (runs: 256, μ: 33761, ~: 33771)
[PASS] test_cannot_setWrongManagementFee(uint256) (runs: 256, μ: 32544, ~: 32474)
[PASS] test_cannot_setWrongPerformanceFee(uint256) (runs: 256, μ: 57485, ~: 57446)
[PASS] test_cannot_transferFromIfNotTransferable() (gas: 251230)
[PASS] test_cannot_transferIfNotTransferable() (gas: 251098)
[PASS] test_cannot_unpauseDeposit_ifNotUnpauser(address) (runs: 256, μ: 27843, ~: 27843)
[PASS] test_cannot_withdrawIfAmountZero() (gas: 36276)
[PASS] test_cannot_withdrawIfConnectorPaused() (gas: 328472)
[PASS] test_cannot_withdrawMoreThanMaxWithdraw(uint256) (runs: 256, μ: 298755, ~: 298696)
[PASS] test_claim_mock(uint256,uint256) (runs: 256, μ: 362742, ~: 364815)
[PASS] test_collectPerformanceFeeWhenDepositing() (gas: 501577)
[PASS] test_collectPerformanceFees(uint256) (runs: 256, μ: 383457, ~: 396600)
[PASS] test_collectPerformanceFeesAfterRemovingFees(uint256) (runs: 256, μ: 410029, ~: 416696)
[PASS] test_collectPerformanceFees_twice(uint256,uint256,uint256,uint256) (runs: 256, μ: 563943,
~: 563948)
[PASS] test_depositAndWithdrawFuzzOffset(uint8,uint256) (runs: 256, μ: 1081245, ~: 1082231)
[PASS] test_deposit_after_protocol_loss(uint256,uint256,uint256,uint256) (runs: 256, μ: 627246,
~: 638580)
[PASS] test_deposit_after_rewards(uint256) (runs: 256, μ: 477404, ~: 477009)
[PASS] test_deposit_rewards_withdraw_dispatch() (gas: 466286)
[PASS] test_initialize(uint256,uint256,bool,string,string) (runs: 256, μ: 5944269, ~: 5937971)
[PASS] test_initializeSetUp() (gas: 51678)
[PASS] test_mint_rewards_redeem_dispatch() (gas: 495552)
[PASS] test_previewDepositIfPaused(uint256) (runs: 256, μ: 94539, ~: 94409)
[PASS] test_previewDeposit_first(uint256) (runs: 256, μ: 59508, ~: 59378)
[PASS] test_previewMintIfPaused(uint256) (runs: 256, μ: 96959, ~: 96620)
[PASS] test_previewMint_first(uint256,uint256) (runs: 256, μ: 78884, ~: 78592)
[PASS] test_previewRedeem_first(uint256) (runs: 256, μ: 50976, ~: 50976)
[PASS] test_previewWithdraw_first(uint256) (runs: 256, μ: 51064, ~: 51064)
[PASS] test_preview_secondAndRewards() (gas: 319671)
[PASS] test_protocol_loss1(uint256) (runs: 256, μ: 530098, ~: 535674)
[PASS] test_protocol_loss_with_collectablePerformanceFees(uint256,uint256,uint256,uint256) (runs:
256, μ: 503894, ~: 507569)
[PASS] test_redeemWithAllowance() (gas: 444896)
[PASS] test_removeFees() (gas: 451249)
[PASS] test_setFeeRecipients() (gas: 69256)
[PASS] test_setManagementFee(uint256) (runs: 256, μ: 39650, ~: 39616)
[PASS] test_setPerformanceFee(uint256) (runs: 256, μ: 62654, ~: 62604)
[PASS] test_setSanctionsList(address) (runs: 256, μ: 33000, ~: 33000)
```

```
[PASS] test_totalAssetsIfPaused() (gas: 305258)
[PASS] test_transferIfTransferable() (gas: 396424)
[PASS] test_withdrawPossibleIfDepositPaused(uint256) (runs: 256, μ: 337949, ~: 337724)
Test result: ok. 66 passed; 0 failed; 0 skipped; finished in 471.76ms

Ran 16 test suites: 151 tests passed, 0 failed, 0 skipped (151 total tests)
```

Code Coverage

Coverage results are close to `100%` for all contracts in scope, which is very good. However, even if tests exist for the contracts `MorphoAaveV2Connector.sol` and `MorphoCompoundV2Connector.sol`, we obtain a coverage of `0.00%`.

Fix Review update

The coverage results decreased a bit for some contracts in scope (Connectors and the Vault), but remains on the overall very good.

File	% Lines	% Statements	% Branches	% Funcs
script/Deploy.goerli.s.sol	0.00% (**0/**78)	0.00% (**0/**89)	0.00% (**0/**22)	0.00% (**0/**13)
script/DeployConfig.goerli.sol	0.00% (**0/**18)	0.00% (**0/**19)	0.00% (**0/**2)	0.00% (**0/**6)
script/utils/Proposer.sol	0.00% (**0/**16)	0.00% (**0/**20)	0.00% (**0/**2)	0.00% (**0/**3)
src/ConnectorRegistry.sol	100.00% (**27/**27)	100.00% (**32/**32)	100.00% (**10/**10)	100.00% (**10/**10)
src/Vault.sol	100.00% (**153/**153)	100.00% (**212/**212)	100.00% (**20/**20)	97.83% (**45/**46)
src/VaultFactory.sol	100.00% (**6/**6)	100.00% (**9/**9)	100.00% (**0/**0)	100.00% (**1/**1)
src/abstracts/FeeDispatcher.sol	100.00% (**65/**65)	100.00% (**90/**90)	90.00% (**18/**20)	100.00% (**12/**12)
src/connectors/AaveV2Connector.sol	100.00% (**5/**5)	100.00% (**7/**7)	100.00% (**0/**0)	100.00% (**3/**3)
src/connectors/AaveV3Connector.sol	100.00% (**5/**5)	100.00% (**7/**7)	100.00% (**0/**0)	100.00% (**3/**3)
src/connectors/CompoundV2Connector.sol	100.00% (**5/**5)	100.00% (**8/**8)	100.00% (**0/**0)	100.00% (**3/**3)
src/connectors/CompoundV3Connector.sol	100.00% (**4/**4)	100.00% (**5/**5)	100.00% (**0/**0)	100.00% (**3/**3)
src/connectors/MorphoAaveV2Connector.sol	0.00% (**0/**7)	0.00% (**0/**8)	100.00% (**0/**0)	0.00% (**0/**3)
src/connectors/MorphoCompoundV2Connector.sol	0.00% (**0/**7)	0.00% (**0/**8)	100.00% (**0/**0)	0.00% (**0/**3)
src/connectors/SDAIConnector.sol	100.00% (**4/**4)	100.00% (**5/**5)	100.00% (**0/**0)	100.00% (**3/**3)
src/proxy/VaultUpgradeableBeacon.sol	100.00% (**18/**18)	100.00% (**20/**20)	100.00% (**4/**4)	100.00% (**8/**8)

File	% Lines	% Statements	% Branches	% Funcs
test/mocks/AssetMock.sol	66.67% (**2**/3)	66.67% (**2**/3)	100.00% (**0**/0)	66.67% (**2**/3)
test/mocks/ConnectorMock.sol	100.00% (**4**/4)	100.00% (**5**/5)	100.00% (**0**/0)	100.00% (**3**/3)
test/mocks/FeeDispatcherMock.sol	100.00% (**5**/5)	100.00% (**5**/5)	100.00% (**0**/0)	100.00% (**5**/5)
test/mocks/ProtocolMock.sol	100.00% (**3**/3)	100.00% (**3**/3)	100.00% (**0**/0)	100.00% (**3**/3)
test/utils/TestHelper.sol	0.00% (**0**/164)	0.00% (**0**/169)	0.00% (**0**/4)	0.00% (**0**/15)
Total	51.26% (**306**/597)	56.63% (**410**/724)	61.90% (**52**/84)	69.80% (**104**/149)

Fix Review update

File	% Lines	% Statements	% Branches	% Funcs
script/Deploy.sepolia.s.sol	0.00% (**0**/78)	0.00% (**0**/89)	0.00% (**0**/22)	0.00% (**0**/13)
script/DeployConfig.sepolia.sol	0.00% (**0**/30)	0.00% (**0**/31)	0.00% (**0**/2)	0.00% (**0**/6)
script/utils/Proposer.sol	0.00% (**0**/16)	0.00% (**0**/20)	0.00% (**0**/2)	0.00% (**0**/3)
src/ConnectorRegistry.sol	100.00% (**27**/27)	100.00% (**36**/36)	100.00% (**12**/12)	100.00% (**11**/11)
src/Vault.sol	99.45% (**181**/182)	98.46% (**255**/259)	92.86% (**39**/42)	100.00% (**54**/54)
src/VaultFactory.sol	100.00% (**6**/6)	100.00% (**8**/8)	100.00% (**0**/0)	100.00% (**1**/1)
src/abstracts/FeeDispatcher.sol	100.00% (**67**/67)	100.00% (**92**/92)	90.00% (**18**/20)	100.00% (**12**/12)
src/connectors/AaveV2Connector.sol	83.33% (**5**/6)	87.50% (**7**/8)	100.00% (**0**/0)	75.00% (**3**/4)
src/connectors/AaveV3Connector.sol	83.33% (**5**/6)	87.50% (**7**/8)	100.00% (**0**/0)	75.00% (**3**/4)
src/connectors/CompoundV2Connector.sol	100.00% (**29**/29)	97.96% (**48**/49)	62.50% (**5**/8)	100.00% (**5**/5)
src/connectors/CompoundV3Connector.sol	80.00% (**4**/5)	83.33% (**5**/6)	100.00% (**0**/0)	75.00% (**3**/4)
src/connectors/MetamorphoConnector.sol	80.00% (**4**/5)	83.33% (**5**/6)	100.00% (**0**/0)	75.00% (**3**/4)
src/connectors/MorphoAaveV2Connector.sol	0.00% (**0**/8)	0.00% (**0**/9)	100.00% (**0**/0)	0.00% (**0**/4)
src/connectors/MorphoBlueConnector.sol	80.00% (**4**/5)	83.33% (**5**/6)	100.00% (**0**/0)	75.00% (**3**/4)

File	% Lines	% Statements	% Branches	% Funcs
src/connectors/MorphoCompoundV2Connector.sol	0.00% (**0/**8)	0.00% (**0/**9)	100.00% (**0/**0)	0.00% (**0/**4)
src/connectors/SDAIConnector.sol	80.00% (**4/**5)	83.33% (**5/**6)	100.00% (**0/**0)	75.00% (**3/**4)
src/connectors/VenusConnector.sol	0.00% (**0/**29)	0.00% (**0/**49)	0.00% (**0/**8)	0.00% (**0/**5)
src/proxy/VaultUpgradeableBeacon.sol	100.00% (**17/**17)	100.00% (**22/**22)	100.00% (**6/**6)	100.00% (**8/**8)
test/mocks/AssetMock.sol	100.00% (**3/**3)	100.00% (**3/**3)	100.00% (**0/**0)	100.00% (**3/**3)
test/mocks/ComptrollerMock.sol	100.00% (**1/**1)	100.00% (**1/**1)	100.00% (**0/**0)	100.00% (**1/**1)
test/mocks/ConnectorMock.sol	100.00% (**11/**11)	100.00% (**13/**13)	66.67% (**4/**6)	100.00% (**4/**4)
test/mocks/FeeDispatcherMock.sol	100.00% (**5/**5)	100.00% (**5/**5)	100.00% (**0/**0)	100.00% (**5/**5)
test/mocks/ProtocolMock.sol	100.00% (**4/**4)	100.00% (**4/**4)	100.00% (**0/**0)	100.00% (**4/**4)
test/mocks/SanctionsListMock.sol	28.57% (**2/**7)	18.18% (**2/**11)	100.00% (**0/**0)	40.00% (**2/**5)
test/mocks/VenusComptrollerMock.sol	0.00% (**0/**1)	0.00% (**0/**1)	100.00% (**0/**0)	0.00% (**0/**1)
test/utils/TestHelper.sol	0.00% (**0/**238)	0.00% (**0/**243)	0.00% (**0/**4)	0.00% (**0/**20)
Total	47.43% (**379/**799)	52.62% (**523/**994)	63.64% (**84/**132)	66.32% (**128/**193)

Changelog

- 2024-02-16 - Initial report
- 2024-02-28 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites&aspo; owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that your access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

Quantstamp